

Julia Algorithms and Optimization

Condensed notes from the unified Pluto notebook

Graphs.jl, DataStructures.jl, Dijkstra, JuMP, duality, sensitivity and progressive LP modelling

Course map. Part I connects graph topology, heaps and shortest-path algorithms. Part II studies a two-variable primal–dual LP and sensitivity. Part III builds a production-planning model progressively, from a basic LP to load balancing, linear-fractional objectives and a scenario MILP.

Contents

1	Part I – Graph algorithms in Julia	2	2.3	Shadow prices and RHS sensitivity	6
1.1	Graphs.jl: topology first	2	2.4	Objective-coefficient sensitivity	7
1.2	Weighted example	2	3	Part III – Progressive LP modelling with JuMP	7
1.3	DataStructures.jl and binary heaps	2	3.1	Data and basic workflow	7
1.4	Dijkstra: one shortest path	4	3.2	Steps 1–3: progressively add linear constraints	7
1.5	All shortest paths	5	3.3	Step 4: load balancing	9
2	Part II – Primal, dual and sensitivity analysis	6	3.4	Step 5: maximize profit per labour hour . .	9
2.1	A two-variable primal–dual pair	6	3.5	Step 6: minimize material per unit of profit	10
2.2	Geometry	6	3.6	Step 7: probabilistic stock reserve	11
			3.7	Matrix versus dictionary data	11
			3.8	Progressive sequence at a glance	12

Part I – Graph algorithms in Julia

Graphs.jl: topology first

Graphs.jl represents vertices, edges/arcs, neighborhoods, degrees and graph algorithms. For a directed graph:

```
using Graphs

G = SimpleDiGraph(5)
add_edge!(G, 1, 2)
add_edge!(G, 1, 3)

nv(G)          # number of vertices
ne(G)          # number of arcs
outneighbors(G, 1) # successors of 1
inneighbors(G, 3) # predecessors of 3
has_edge(G, 1, 2)
```

For Dijkstra, the central graph operation is

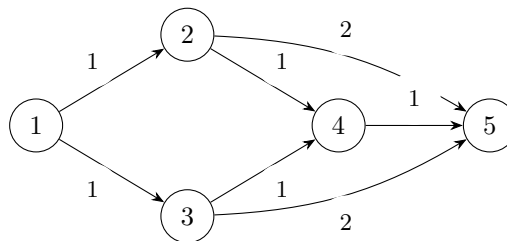
```
for v in outneighbors(G, u)
    # relax arc (u,v)
end
```

Weighted example

The notebook stores arc lengths separately in a dictionary:

```
w = Dict{
    (1,2) => 1.0, (1,3) => 1.0,
    (2,4) => 1.0, (3,4) => 1.0,
    (2,5) => 2.0, (3,5) => 2.0,
    (4,5) => 1.0,
}

G = SimpleDiGraph(5)
for (u,v) in keys(w)
    add_edge!(G, u, v)
end
```



For built-in Graphs.jl shortest-path routines, the same weights can be stored in a matrix:

```
distmx = fill(Inf, nv(G), nv(G))
for ((u,v), c) in w
    distmx[u,v] = c
end
```

DataStructures.jl and binary heaps

Dijkstra repeatedly needs the unprocessed vertex with the smallest tentative distance. A min-heap supports that operation efficiently.

```
using DataStructures

H = BinaryMinHeap{Tuple{Float64,Int}}{Int}()
push!(H, (5.0, 3))
push!(H, (2.0, 2))
push!(H, (0.0, 1))

first(H) # minimum pair
```

```
pop!(H)          # remove and return the minimum  
isempty(H)
```

Julia compares tuples lexicographically, so a heap of `(distance, vertex)` pairs extracts the smallest distance first.

A `BinaryMinHeap` is not intended to be traversed with `collect(H)`. Access its root with `first(H)` and repeatedly extract with `pop!(H)` when needed.

Dijkstra: one shortest path

For a source s , initialize $d_s = 0$, $d_v = +\infty$ for $v \neq s$, repeatedly extract the smallest tentative distance, and relax outgoing arcs. Arc lengths must be nonnegative.

```
function dijkstra_one_path(G, w, source)
    n = nv(G)
    dist = fill(Inf, n)
    parent = fill(0, n)
    dist[source] = 0.0

    H = BinaryMinHeap{Tuple{Float64,Int}}{Int}()
    push!(H, (0.0, source))

    while !isempty(H)
        du, u = pop!(H)
        du > dist[u] && continue

        for v in outneighbors(G, u)
            newdist = du + w[(u,v)]
            if newdist < dist[v]
                dist[v] = newdist
                parent[v] = u
                push!(H, (newdist, v))
            end
        end
    end
    return dist, parent
end
```

The line

```
du > dist[u] && continue
```

implements a *lazy heap update*: when a distance improves, a new pair is inserted rather than modifying the old heap entry. Obsolete entries are ignored when extracted.

One path is reconstructed by following the parent array backward:

```
function reconstruct_path(parent, source, target)
    target == source && return [source]
    parent[target] == 0 && return Int[]

    path = Int[]
    v = target
    while v != 0
        push!(path, v)
        v == source && break
        v = parent[v]
    end
    reverse!(path)
    return path
end
```

Built-in comparison:

```
state = dijkstra_shortest_paths(G, 1, distmx)
state.dists
state.parents
enumerate_paths(state, 5)
```

When several shortest paths exist, two correct implementations may return different paths. Compare the shortest distance and the length of the returned path, not necessarily the vertex sequence.

All shortest paths

A single parent loses ties. Replace `parent[v]::Int` by a vector of predecessors `pred[v]::Vector{Int}`.

For an arc (u, v) :

$$\begin{cases} \text{if } d_u + w_{uv} < d_v : & d_v \leftarrow d_u + w_{uv}, \quad \text{pred}(v) \leftarrow \{u\}, \\ \text{if } d_u + w_{uv} = d_v : & \text{pred}(v) \leftarrow \text{pred}(v) \cup \{u\}. \end{cases}$$

```
function dijkstra_all_predecessors(G, w, source)
    n = nv(G)
    dist = fill{Inf, n}
    pred = [Int[] for _ in 1:n]
    dist[source] = 0.0

    H = BinaryMinHeap{Tuple{Float64,Int}}{Int}()
    push!(H, (0.0, source))

    while !isempty(H)
        du, u = pop!(H)
        du > dist[u] && continue

        for v in outneighbors(G, u)
            newdist = du + w[(u,v)]
            if newdist < dist[v]
                dist[v] = newdist
                pred[v] = [u]
                push!(H, (newdist, v))
            elseif newdist == dist[v]
                push!(pred[v], u)
            end
        end
    end
    return dist, pred
end
```

The predecessor lists compactly represent the shortest-path predecessor graph. Explicit paths are then enumerated recursively:

```
function enumerate_shortest_paths(pred, source, target)
    target == source && return [[source]]
    isempty(pred[target]) && return Vector{Vector{Int}}{Int}()

    paths = Vector{Vector{Int}}{Int}()
    for u in pred[target]
        for p in enumerate_shortest_paths(pred, source, u)
            push!(paths, [p; target])
        end
    end
    return paths
end
```

For the example, the shortest distance from 1 to 5 is 3 and there are four shortest paths:

$$1 \rightarrow 2 \rightarrow 5, \quad 1 \rightarrow 3 \rightarrow 5, \quad 1 \rightarrow 2 \rightarrow 4 \rightarrow 5, \quad 1 \rightarrow 3 \rightarrow 4 \rightarrow 5.$$

Built-in all-paths mode:

```
state = dijkstra_shortest_paths(G, 1, distmx; allpaths=true)
state.predecessors
state.pathcounts
enumerate_shortest_paths(state.predecessors, 1, 5)
```

With adjacency lists and a binary heap, Dijkstra runs in $O((n + m) \log n)$. However, the number of shortest paths may be exponential, so explicitly listing all of them cannot in general be polynomial in the input size. The recursive enumeration is simplest with strictly positive arc lengths; zero-length cycles require cycle protection if only simple paths are desired.

Part II – Primal, dual and sensitivity analysis

A two-variable primal–dual pair

Primal

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 4, \\ & 2x_1 + x_2 \leq 6, \\ & x_1, x_2 \geq 0. \end{aligned}$$

Dual

$$\begin{aligned} \min \quad & 4y_1 + 6y_2 \\ \text{s.t.} \quad & y_1 + 2y_2 \geq 3, \\ & y_1 + y_2 \geq 2, \\ & y_1, y_2 \geq 0. \end{aligned}$$

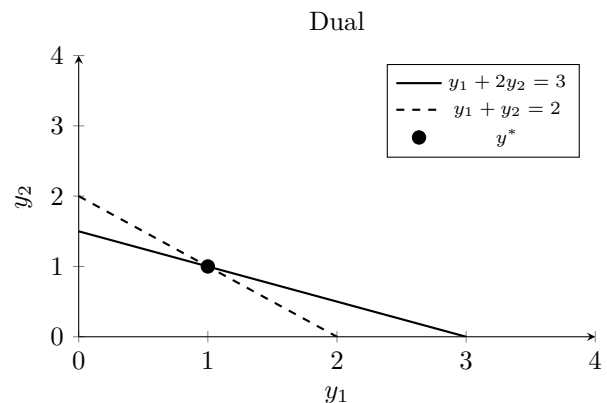
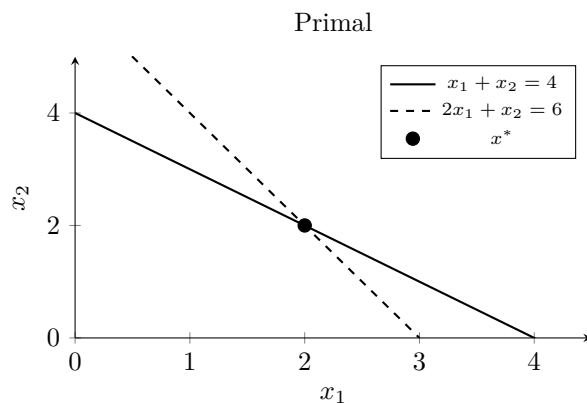
```
function solve_primal(b1=4.0, b2=6.0)
    model = Model(HiGHS.Optimizer)
    set_silent(model)
    @variable(model, x[1:2] >= 0)
    @objective(model, Max, 3x[1] + 2x[2])
    c1 = @constraint(model, x[1] + x[2] <= b1)
    c2 = @constraint(model, 2x[1] + x[2] <= b2)
    optimize!(model)
    return value.(x), objective_value(model),
           [shadow_price(c1), shadow_price(c2)]
end
```

```
function solve_dual(c1=3.0, c2=2.0)
    model = Model(HiGHS.Optimizer)
    set_silent(model)
    @variable(model, y[1:2] >= 0)
    @objective(model, Min, 4y[1] + 6y[2])
    @constraint(model, y[1] + 2y[2] >= c1)
    @constraint(model, y[1] + y[2] >= c2)
    optimize!(model)
    return value.(y), objective_value(model)
end
```

The solutions are

$$x^* = (2, 2), \quad y^* = (1, 1), \quad z_P^* = z_D^* = 10.$$

Geometry



Shadow prices and RHS sensitivity

While the optimal basis remains unchanged, the dual variables are the marginal values of the primal right-hand sides:

$$y_1^* = \frac{\partial z^*}{\partial b_1}, \quad y_2^* = \frac{\partial z^*}{\partial b_2}.$$

If both resource constraints stay active,

$$x_1 + x_2 = b_1, \quad 2x_1 + x_2 = b_2,$$

hence

$$x_1 = b_2 - b_1, \quad x_2 = 2b_1 - b_2.$$

Variation	Same basis feasible	Optimal value
b_1 with $b_2 = 6$	$3 \leq b_1 \leq 6$	$z^* = 6 + b_1$
b_2 with $b_1 = 4$	$4 \leq b_2 \leq 8$	$z^* = 4 + b_2$

Therefore both marginal values are 1 in these intervals, in agreement with $y^* = (1, 1)$.

Objective-coefficient sensitivity

At $x^* = (2, 2)$:

- keeping $c_2 = 2$, the same vertex remains optimal for $2 \leq c_1 \leq 4$;
- keeping $c_1 = 3$, it remains optimal for $\frac{3}{2} \leq c_2 \leq 3$.

At an endpoint, alternative optimal solutions occur along an edge.

Part III – Progressive LP modelling with JuMP

Data and basic workflow

The production example has three products: Standard, Premium and Eco.

Product	Unit profit	Maximum demand
Standard	38	80
Premium	62	55
Eco	50	70

The basic JuMP workflow is:

```
model = Model(HiGHS.Optimizer)
@variable(model, ...)
@constraint(model, ...)
@objective(model, Max, ...)
optimize!(model)
```

Steps 1–3: progressively add linear constraints

Step 1: demand only.

$$\max \sum_{p \in P} c_p q_p \quad \text{s.t.} \quad 0 \leq q_p \leq d_p \quad (p \in P).$$

Step 2: stock constraints. Resources are Steel, Electronics and Packaging.

Resource	Standard	Premium	Eco	Stock
Steel	2.0	3.2	1.6	260
Electronics	0.8	1.6	1.1	140
Packaging	1.0	1.4	1.8	230

For each resource r :

$$\sum_{p \in P} a_{rp} q_p \leq S_r.$$

Step 3: working-time constraints.

Department	Standard	Premium	Eco	Available hours
Assembly	1.1	1.9	1.4	165
Finishing	0.8	1.3	1.5	130
Quality control	0.5	0.9	0.8	95

For each department d :

$$\sum_{p \in P} h_{dp} q_p \leq H_d.$$

A compact Step-3 JuMP model is therefore:

```
P = eachindex(data.product_names)
R = eachindex(data.resource_names)
D = eachindex(data.department_names)

model = Model(HiGHS.Optimizer)
@variable(model, q[p in P] >= 0)
@constraint(model, demand[p in P], q[p] <= data.demand[p])
@constraint(model, stock[r in R],
```

```
sum(data.resource_use[r,p] * q[p] for p in P) <= data.stock[r])
@constraint(model, hours[d in D],
    sum(data.hours_use[d,p] * q[p] for p in P) <= data.hours_available[d])
@objective(model, Max, sum(data.profit[p] * q[p] for p in P))
optimize!(model)
```

A recurring modelling pattern is visible: an index set plus a coefficient matrix generates an entire family of linear constraints. The same JuMP syntax models stock, working time and many other aggregate resource limits.

Step 4: load balancing

Three teams have different capacities, so balancing absolute hours is inappropriate. Balance their utilization rates instead:

$$L_t = \sum_{p \in P} b_{tp} q_p, \quad u_t = \frac{L_t}{C_t}.$$

Team	Standard	Premium	Eco	Capacity
A	1.2	1.6	1.0	150
B	0.7	1.2	1.7	140
C	0.9	0.8	1.3	120

Let $\bar{u}$ be average utilization. Minimize the maximum deviation Δ :

$$L_t = C_t u_t, \quad u_t - \bar{u} \leq \Delta, \quad \bar{u} - u_t \leq \Delta.$$

To retain profitability:

$$\sum_{p \in P} c_p q_p \geq 0.95 Z^*,$$

where Z^* is the optimal profit from Step 3.

Essential JuMP additions:

```
@variable(model, util[t in T] >= 0)
@variable(model, average_util >= 0)
@variable(model, delta >= 0)

@expression(model, team_load[t in T],
    sum(data.team_hours[t,p] * q[p] for p in P))

@constraint(model, utilization[t in T],
    team_load[t] == data.team_capacity[t] * util[t])
@constraint(model, avg,
    average_util == sum(util[t] for t in T) / length(T))
@constraint(model, upper[t in T], util[t] - average_util <= delta)
@constraint(model, lower[t in T], average_util - util[t] <= delta)
@constraint(model, profit_floor,
    sum(data.profit[p] * q[p] for p in P) >= 0.95 * Zstar)
@objective(model, Min, delta)
```

Step 5: maximize profit per labour hour

Define

$$P(q) = \sum_p c_p q_p, \quad L(q) = \sum_p \ell_p q_p,$$

and consider

$$\max \frac{P(q)}{L(q)}.$$

The Charnes–Cooper transformation introduces

$$\tau = \frac{1}{L(q)}, \quad z_p = \tau q_p.$$

Then

$$\sum_p \ell_p z_p = 1, \quad Az \leq b\tau,$$

and the objective becomes

$$\max \sum_p c_p z_p.$$

Essential JuMP pattern:

```
@variable(model, z[p in P] >= 0)
@variable(model, tau >= 0)

@constraint(model, normalization,
    sum(ell[p] * z[p] for p in P) == 1)
```

```

@constraint(model, demand[p in P], z[p] <= data.demand[p] * tau)
@constraint(model, stock[r in R],
    sum(data.resource_use[r,p] * z[p] for p in P) <= data.stock[r] * tau)
@constraint(model, hours[d in D],
    sum(data.hours_use[d,p] * z[p] for p in P) <= data.hours_available[d] * tau)
@objective(model, Max, sum(data.profit[p] * z[p] for p in P))

```

The transformed model determines an optimal production direction; the notebook then computes the largest feasible scale to display concrete quantities.

Step 6: minimize material per unit of profit

Let

$$M(q) = \sum_p m_p q_p, \quad P(q) = \sum_p c_p q_p,$$

and minimize

$$\frac{M(q)}{P(q)}.$$

Use

$$\tau = \frac{1}{P(q)}, \quad z_p = \tau q_p,$$

so that

$$\sum_p c_p z_p = 1, \quad \min \sum_p m_p z_p.$$

The scaled feasibility constraints are identical to Step 5. This illustrates how one transformation can be reused for different linear-fractional objectives.

Step 7: probabilistic stock reserve

Steel is treated as critical. The model requires:

1. hard steel stock feasibility in every scenario;
2. steel usage at most 90% of stock in at least 95% of the scenarios.

The chance constraint is

$$\Pr(\text{steel consumption} \leq 0.90 \times \text{steel stock}) \geq 0.95.$$

With scenario multipliers γ_s , impose hard feasibility

$$\sum_p \gamma_s a_{rp} q_p \leq S_r \quad (s \in S),$$

and allow reserve violations using binary variables y_s :

$$\sum_p \gamma_s a_{rp} q_p \leq 0.90 S_r + M_s y_s.$$

For 20 equally likely scenarios, a 95% requirement permits at most one violation:

$$\sum_{s \in S} y_s \leq 1.$$

The notebook also normalizes production by steel stock:

$$w_p = \frac{q_p}{S_r},$$

so the reserve condition becomes

$$\sum_p \gamma_s a_{rp} w_p \leq 0.90 + \widetilde{M}_s y_s.$$

Because the denominator is a constant, this is a normalization analogous to Charnes–Cooper, not a full linear-fractional transformation.

Core JuMP structure:

```
@variable(model, q[p in P] >= 0)
@variable(model, w[p in P] >= 0)
@variable(model, reserve_violation[s in S], Bin)

@constraint(model, scaled[p in P], q[p] == stockcrit * w[p])
@constraint(model, hard[s in S],
    sum(gamma[s] * a[p] * q[p] for p in P) <= stockcrit)
@constraint(model, reserve[s in S],
    sum(gamma[s] * a[p] * w[p] for p in P)
    <= 0.90 + big_m[s] * reserve_violation[s])
@constraint(model, chance,
    sum(reserve_violation[s] for s in S) <= allowed_violations)
```

Matrix versus dictionary data

The notebook mainly uses matrices because they are compact and close to mathematical notation:

```
resource_use[r,p]
hours_use[d,p]
team_hours[t,p]
```

A dictionary can be more explicit for irregular or externally imported sets:

```
resource_use_dict[("Steel", "Premium")]
```

For introductory LP modelling, the matrix representation is usually the clearer starting point once row and column meanings are stated.

Progressive sequence at a glance

Step	New modelling concept	New JuMP concept
1	production and demand	variables, indexed constraints, objective
2	stock limits	matrix coefficients, resource-indexed constraints
3	working-time limits	another indexed constraint family
4	load balancing	expressions, auxiliary variables, absolute-value linearization
5	profit per labour hour	Charnes–Cooper for a maximization ratio
6	material per profit	Charnes–Cooper for a minimization ratio
7	probabilistic reserve	scenario MILP, binaries, normalized stock usage

Overall take-away. In Julia, a mathematical algorithm is often assembled from complementary components: domain structures (`Graphs.jl`), efficient data structures (`DataStructures.jl`), modelling syntax (`JuMP`), and numerical solvers (`HiGHS`). The modelling workflow is equally progressive: formulate the mathematical quantity first, then introduce variables and constraints, and finally translate them into `JuMP` syntax.